# Hierarchical Context Merging: Better Long Context Understanding for Pre-trained LLMs

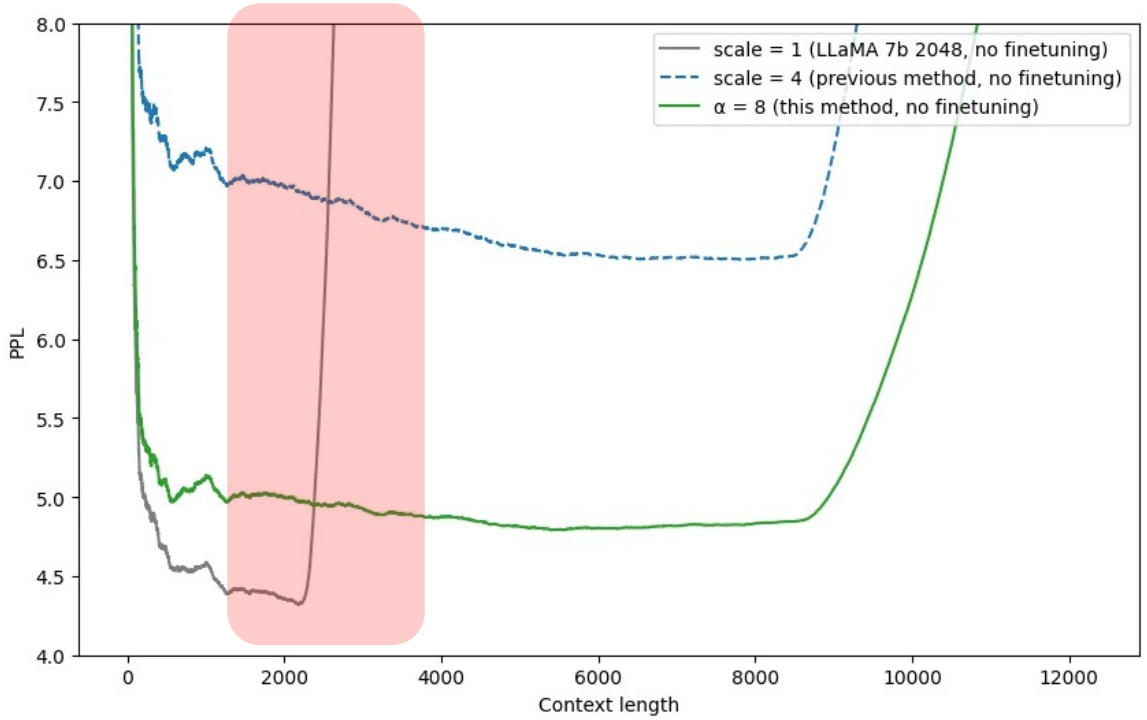
Woomin Song[1,*], Seunghyuk Oh[1,*], Sangwoo Mo[2], Jaehyung Kim[3],
Sukmin Yun[4], Jung-Woo Ha[5], Jinwoo Shin[1]

*ICLR 2024*

[1] KAIST, [2] University of Michigan, [3] CMU, [4] Hanyang University ERICA , [5] NAVER

# Context Limit of LLMs

- **Context limit**: maximum number of input tokens that LLMs can handle




*Degradation of Perplexity (PPL) from Llama[1]*

[1] https://www.reddit.com/r/LocalLLaMA/comments/14lz7j5/ntkaware_scaled_rope_allows_llama_models_to_have/

# Reason of Context Limit

- **Why it occurs?** Due to fixed maximum input lengths during training
  - *e.g.*, 4k tokens for Llama-2[1]
  - Hence, LLMs are struggle with generalization far beyond these lengths (OOD)

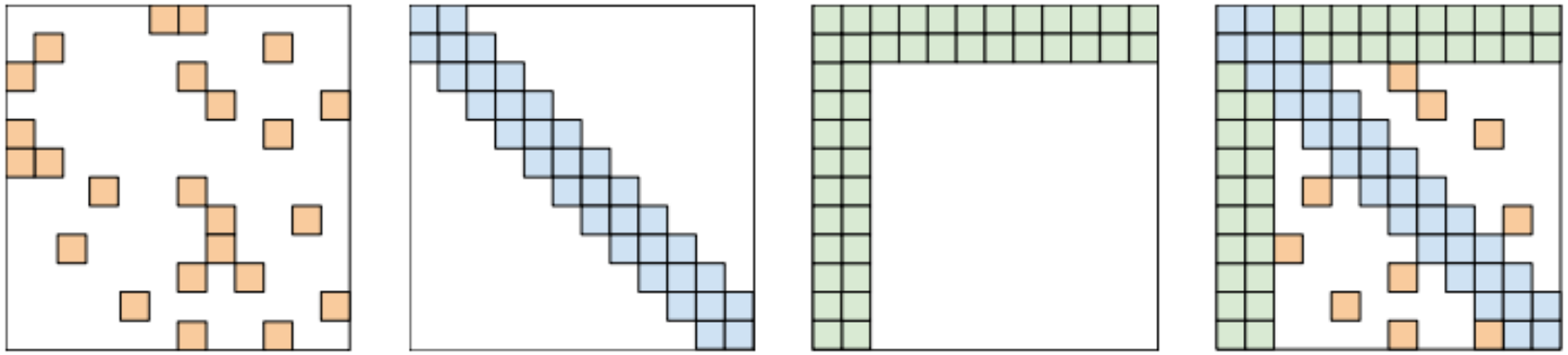
| | Training Data | Params | Context Length | GQA | Tokens | LR |
|---|---|---|---|---|---|---|
| Llama 1 | *See Touvron et al. (2023)* | 7B | 2k | ✗ | 1.0T | $3.0 \times 10^{-4}$ |
| | | 13B | 2k | ✗ | 1.0T | $3.0 \times 10^{-4}$ |
| | | 33B | 2k | ✗ | 1.4T | $1.5 \times 10^{-4}$ |
| | | 65B | 2k | ✗ | 1.4T | $1.5 \times 10^{-4}$ |
| Llama 2 | *A new mix of publicly available online data* | 7B | 4k | ✗ | 2.0T | $3.0 \times 10^{-4}$ |
| | | 13B | 4k | ✗ | 2.0T | $3.0 \times 10^{-4}$ |
| | | 34B | 4k | ✓ | 2.0T | $1.5 \times 10^{-4}$ |
| | | 70B | 4k | ✓ | 2.0T | $1.5 \times 10^{-4}$ |

*Details of Llama variants[1]*

[1] Touvron et al., Llama 2: Open Foundation and Fine-Tuned Chat Models., Meta AI

# Challenges of Training with Long Inputs

- **Why limited input length?** Due to computational cost of Transformer[1]
  - Quadratic computational nature of self-attention with respect to **input length**
  - Efficient Transformer variants[2,3] have been explored, but not very successful yet
  - Also, collecting datasets with such long input itself is also challenging problem




*Sparse attention variants[2]*

[1] Vaswani et al., Attention Is All You Need., NeurIPS 2017
[2] Zaheer et al., Big Bird: Transformers for Longer Sequences., NeurIPS 2020
[3] Wang et al., Linformer: Self-Attention with Linear Complexity., arXiv:2006

# Extending Context Limit of Pre-trained LLMs

- **Recent focus**: frameworks to extend context limit of pre-trained LLMs
  - *How?* By modifying (or interpolating) **positional encoding**
  - *e.g.*, linear interpolation[1] or non-linear interpolation[2,3]

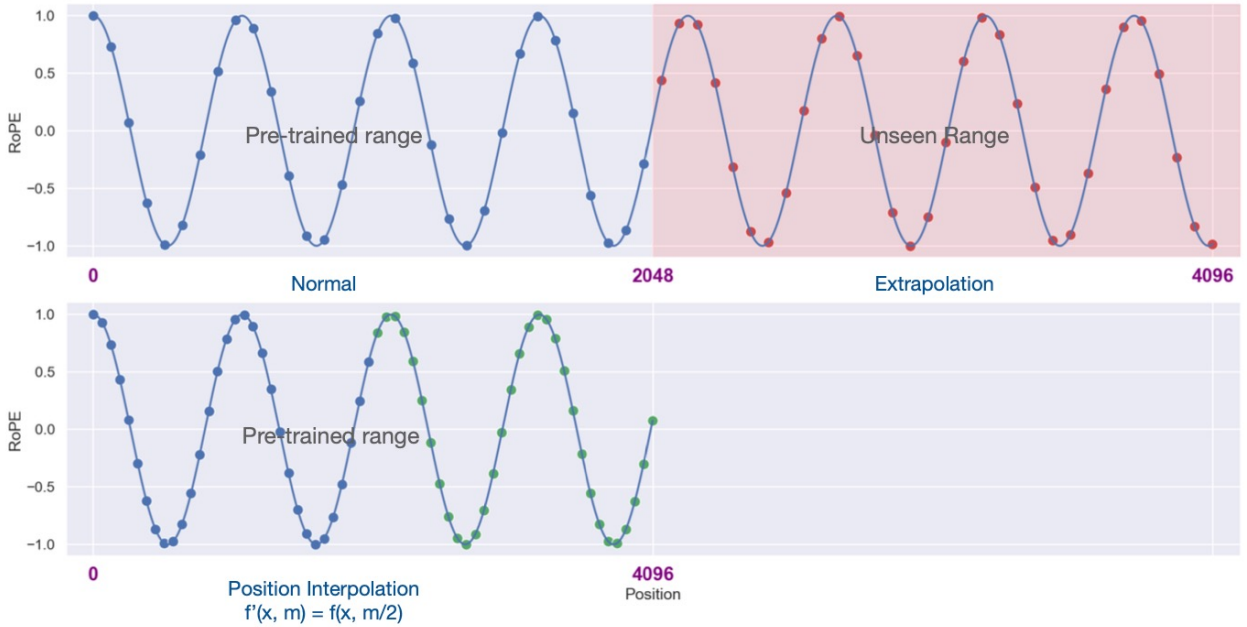



*Illustration of positional encoding interpolation[1]*


[1] Chen et al., Extending Context Window of Large Language Models via Positional Interpolation., arXiv:23.06
[2] https://www.reddit.com/r/LocalLLaMA/comments/14lz7j5/ntkaware_scaled_rope_allows_llama_models_to_have/
[3] Peng et al., YaRN: Efficient Context Window Extension of Large Language Models., ICLR 2024

# Limitation of Interpolation-based Method

- However, these methods still require **quadratic computational cost**
  - *Namely,* reduction of computation is also crucial to handle long input context

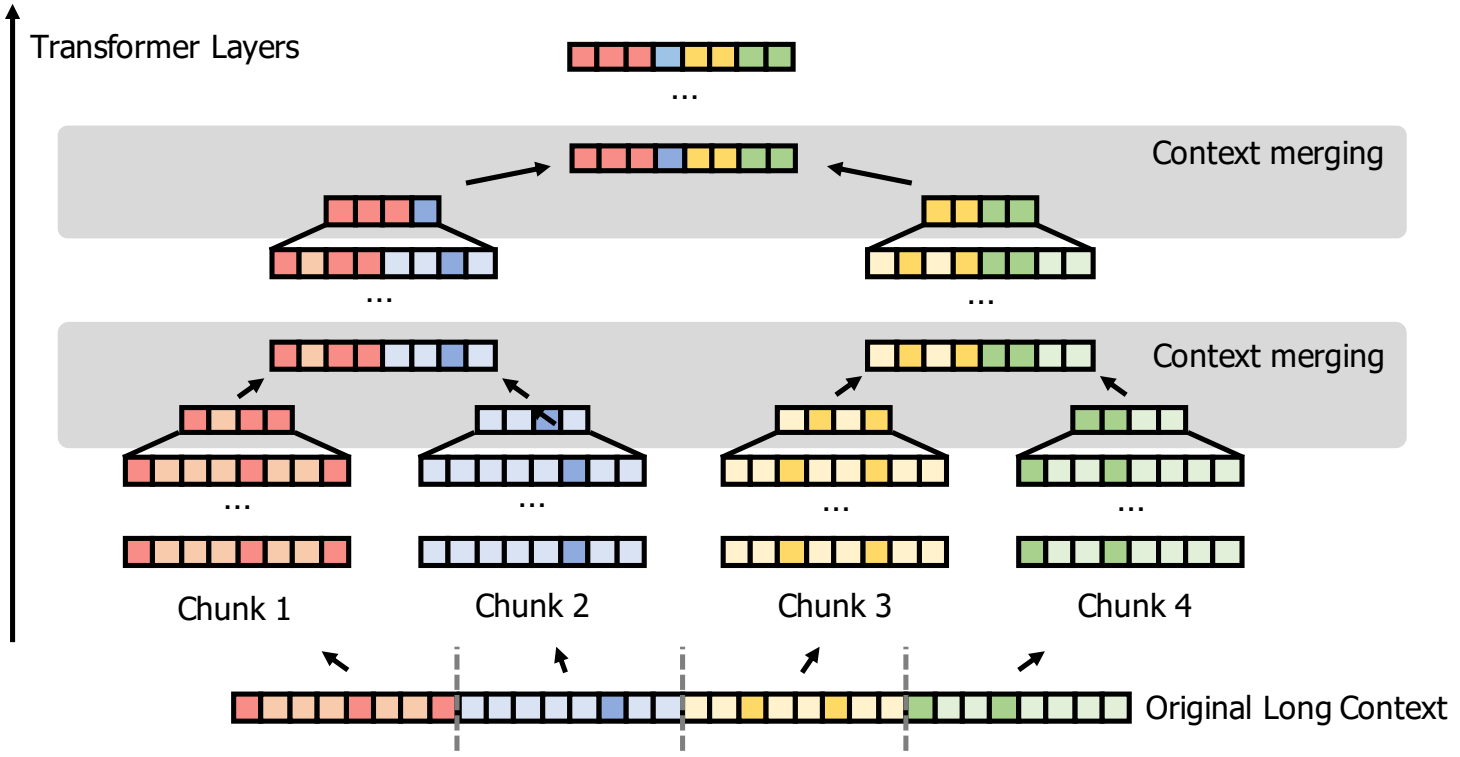
# Limitation of Interpolation-based Method

- However, these methods still require **quadratic computational cost**
  - *Namely,* reduction of computation is also crucial to handle long input context

🧐 Goal: develop a framework that could extend context limit and also ensure computational efficiency?

# Proposed Method: HOMER

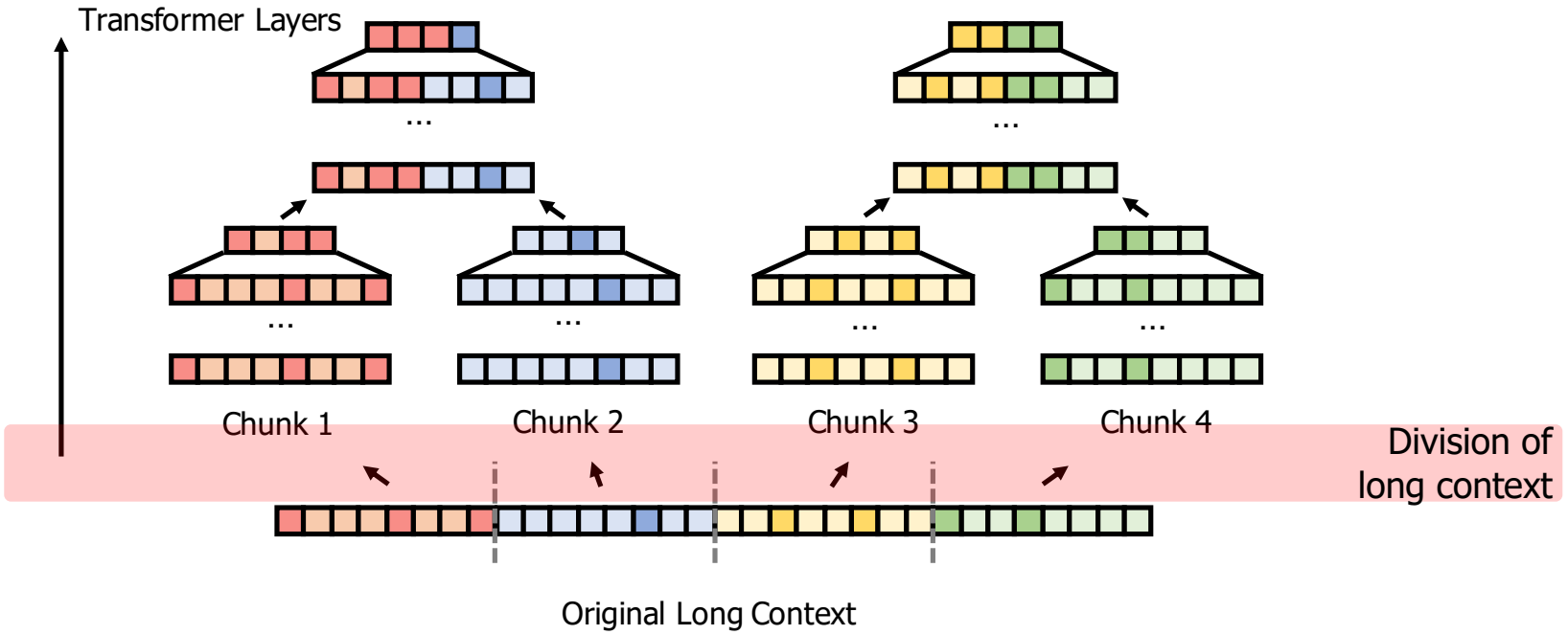
- HOMER: Enlarge context limit of LLM via **H**ierarchical c**O**ntext **MER**ging
  - Key idea: Divide-and-conquer of context + token reduction to remove redundancy




Hierarchical merging of context embeddings

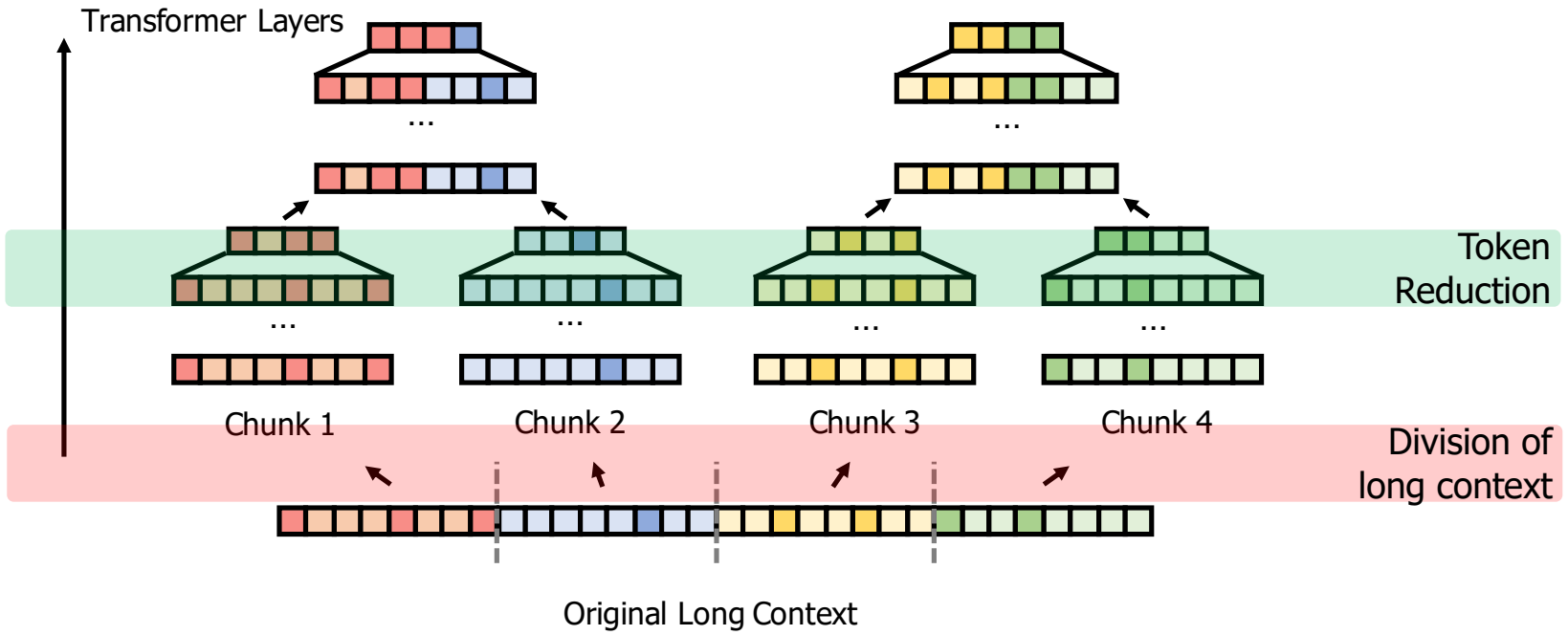
# Proposed Method: HOMER

- Specifically, HOMER conducts following steps for **hierarchical merging**
  1. Division of input (long) context into multiple chunks



# Proposed Method: HOMER

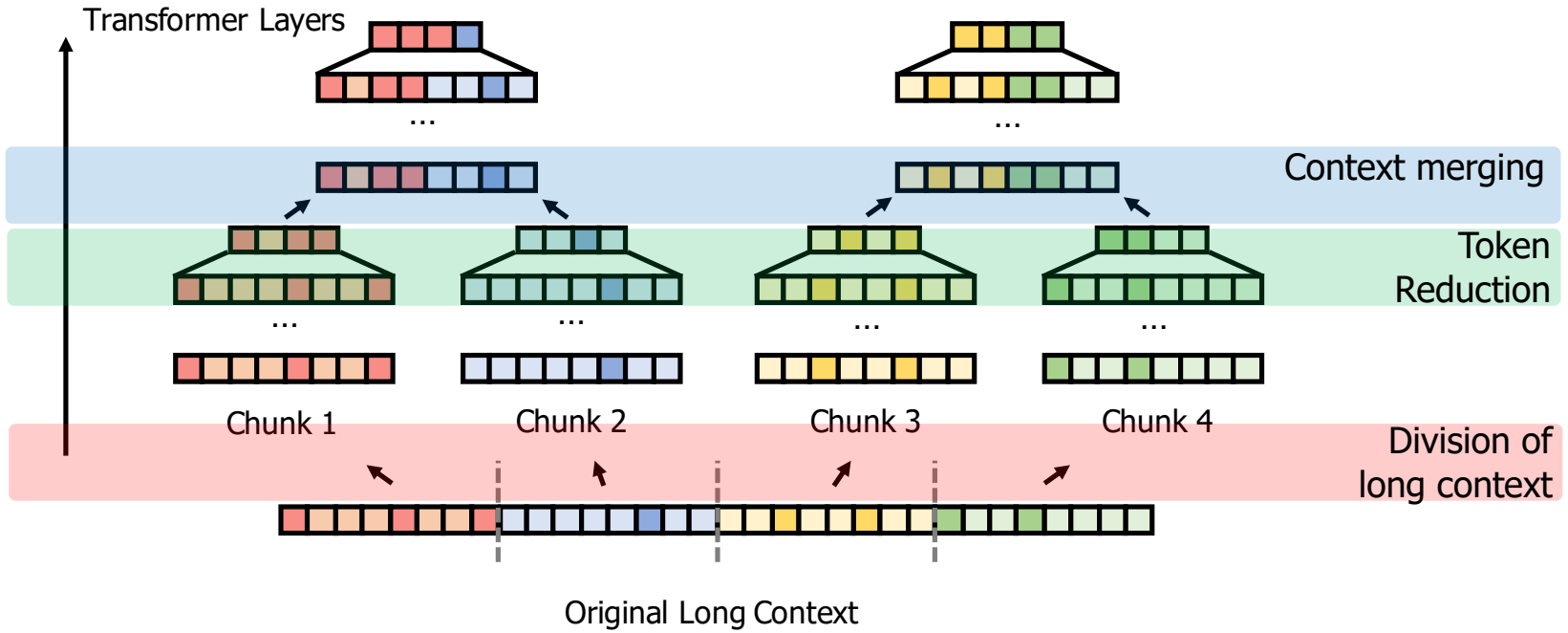
- Specifically, HOMER conducts following steps for **hierarchical merging**
  1. Division of input (long) context into multiple chunks
  2. Token reduction on individual chunks



# Proposed Method: HOMER

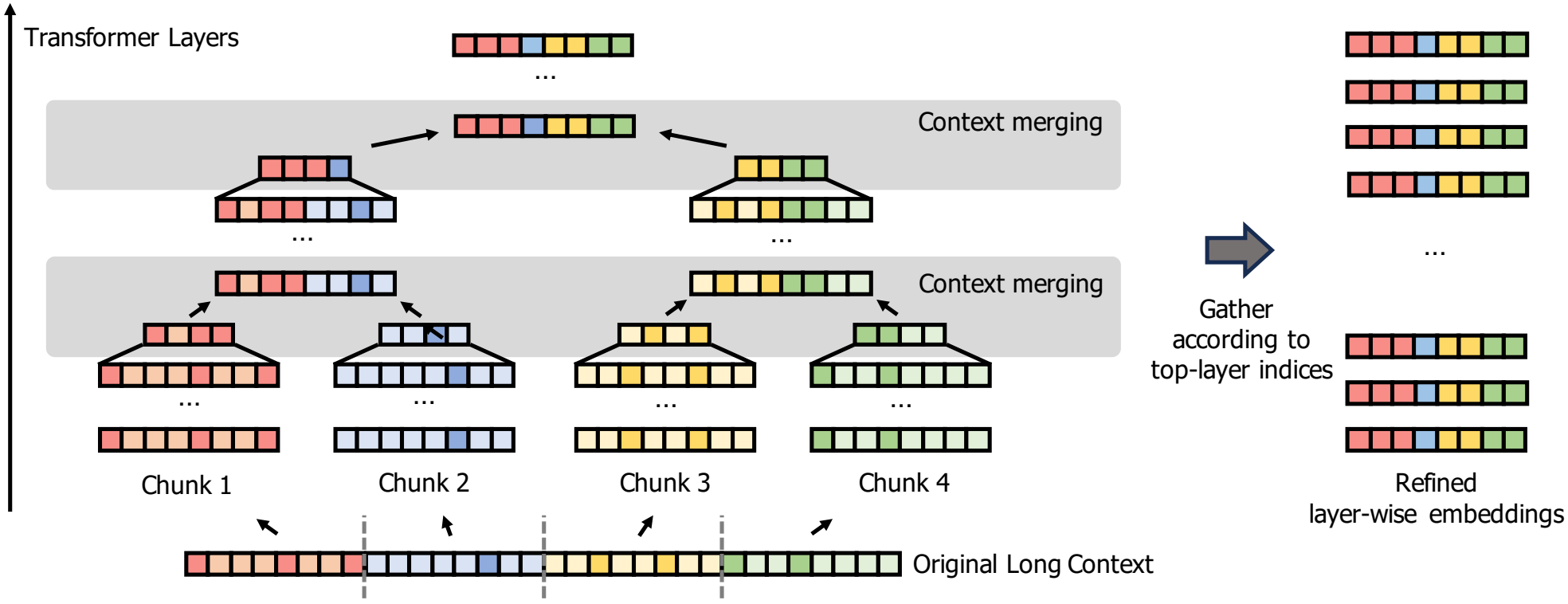
- Specifically, HOMER conducts following steps for **hierarchical merging**
  1. Division of input (long) context into multiple chunks
  2. Token reduction on individual chunks
  3. Merging embeddings of adjacent chunks

Iterated for each Transformer layer



# Proposed Method: HOMER

- Next, HOMER **refine lower-layer** embeddings based on merged context

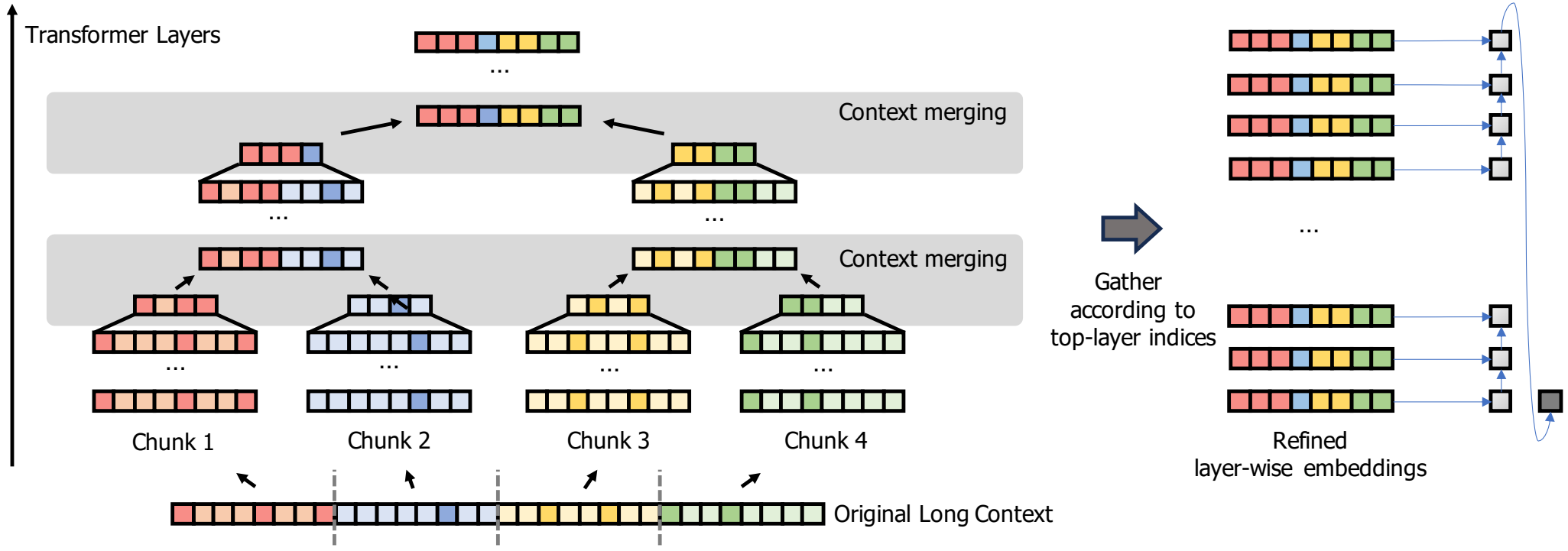



Hierarchical merging of context embeddings

Propagative refinement

# Proposed Method: HOMER

- Next, HOMER **refine lower-layer** embeddings based on merged context
  - Then, LLM could generate next tokens with reduced context and computations




Hierarchical merging of context embeddings

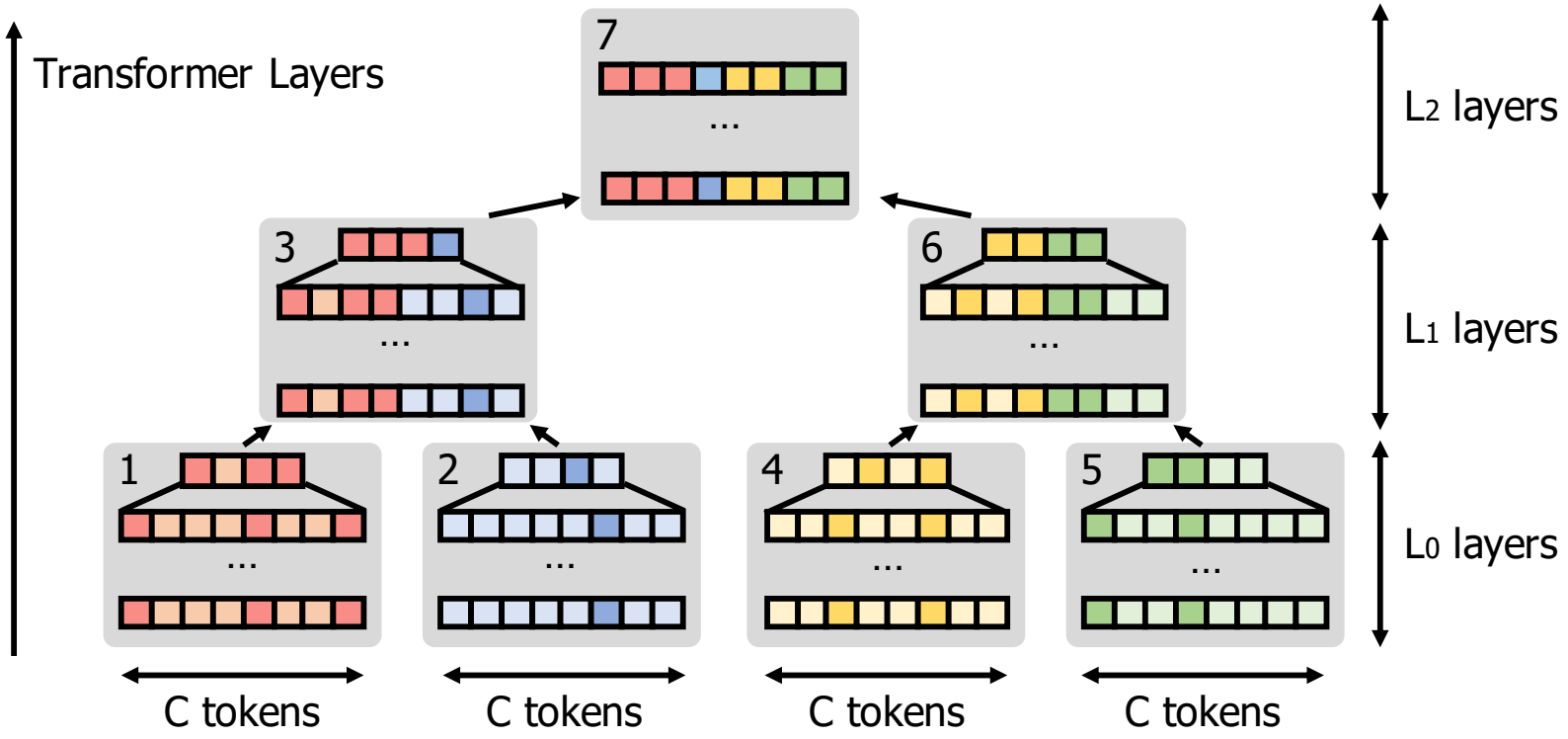
Propagative refinement

Next token generation

# HOMER: Computation Order Optimization

- HOMER → Traversal on binary tree from leaves to root
  - Key idea: (Reduce & Merge + Propagate refinement) with Depth-first search (DFS)

    ⟹ Computational memory increases **logarithmically** with respect to **input length**



# Experiments: Performance

- Three different tasks for experiments
  1. Passkey retrieval
  2. Question answering (QA) on long documents
  3. Language modeling conditioned on long documents

- Llama-2[1] is mainly considered LLM
  - (1) Language modeling: vanilla model, (2) passkey retrieval & QA: chat model
  - Baselines: training-free approaches to extend context limit (interpolation-based)

[1] Touvron et al., Llama 2: Open Foundation and Fine-Tuned Chat Models., Meta AI

# Experiments: Computational Efficiency

- HOMER **effectively reduces** both peak memory and inference time
  - Setup: all measurements are taken on a single A100 GPU, with Flash Attention 2[1]

| | Peak GPU memory (GB) | | | | |
|---|---|---|---|---|---|
| Setup | 4k | 8k | 16k | 32k | 64k |
| Baselines | 18.2 | 23.6 | 34.4 | 56.7 | > 80 |
| HOMER | 16.3 (-10.8%) | 16.5 (-30.1%) | 16.7 (-51.4%) | 17.6 (-68.9%) | 21.3 (at least -73.4%) |
| + Baselines | 19.2 (+5.5%) | 20.1 (-14.6%) | 20.3 (-41.1%) | 20.8 (-63.4%) | 22.5 (at least -71.8%) |

| | Average run time (seconds) | | |
|---|---|---|---|
| Setup | 8k | 16k | 32k |
| ***20 tokens*** | | | |
| Baselines* | 1.879 | 3.224 | 6.546 |
| HOMER | 1.673 (12.3% speedup) | 2.270 (42.0% speedup) | 3.513 (86.3% speedup) |
| ***50 tokens*** | | | |
| Baselines* | 3.842 | 6.028 | 11.143 |
| HOMER | 3.026 (27.0% speedup) | 3.639 (65.6% speedup) | 4.873 (128.7% speedup) |
| ***100 tokens*** | | | |
| Baselines* | 7.149 | 10.733 | 18.828 |
| HOMER | 5.355 (33.5% speedup) | 5.930 (81.0% speedup) | 7.169 (162.6% speedup) |

[1] Dao, FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning., ICLR 2024

Thank you for attention 😄